

TURNITIN OFFICIAL

RecallChain Research Paper.docx

 Adarsh Intermediate Academy

Document Details

Submission ID

trn:oid::7:639518553

Submission Date

Oct 4, 2026, 10:10 AM GMT

Download Date

Oct 4, 2026, 10:11 AM GMT

File Name

RecallChain Research Paper.docx

File Size

374.8 KB

8 Pages**5,740 Words****32,825 Characters**

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

RecallChain: A Hybrid Blockchain-Based Pharmaceutical Supply Chain Provenance Framework for Traceability, Product Authentication, and Accountable Recall

Abstract— Falsified and substandard medicines remain a persistent public-health threat, and most blockchain-based pharmaceutical traceability systems stop at batch-level custody records, leaving individual packs unauthenticated and recalls unverifiable. This paper presents RecallChain, a hybrid on-chain/off-chain framework that adds three capabilities to a role-based batch pipeline: (i) unit-level scratch-code authentication committed through a single Merkle root per batch; (ii) wallet-free, privacy-preserving unit claims, relayed with the patient's signature, whose state turns repeated scans into a clone and diversion signal; and (iii) manufacturer-initiated recalls in which every handler must quarantine stock and confirm before an on-chain deadline, with a closure report that accounts for every pack. The prototype comprises a Solidity contract, an Express relay and a Next.js client. A 33-test suite on a local Hardhat network passed in full, and a gas snapshot shows constant-cost batch registration (204,006 gas on average) and about 89,000 gas per claim. The evaluation is functional and local: it measures neither latency, throughput nor real-world detection rates, and statistical anomaly detection is not implemented. We state the threat model, the residual risks (for example self-reported recall actions and front-running of claims) and the experiments required before field deployment.

Keywords— pharmaceutical supply chain, blockchain, smart contracts, Merkle tree, product authentication, drug recall, traceability, counterfeit medicines

I. INTRODUCTION

Substandard and falsified medical products are a continuing threat to patient safety. The World Health Organization reports that at least one in ten medicines in low- and middle-income countries are substandard or falsified, and that countries spend an estimated US\$30.5 billion per year on such products [1]. Regulators have responded with serialization regimes that require machine-readable identifiers on packs: the U.S. Drug Supply Chain Security Act [2], the European Union's delegated regulation on safety features [3] and, in India, the bar-code or QR-code requirement for listed drug brands introduced by G.S.R. 823(E) [4].

Blockchain technology has been widely proposed as the integrity layer for such regimes. A systematic review of 38 studies found counterfeit prevention to be the most common application domain, at 45% of the papers [5], and a 2026 mapping review of 103 records describes a concentration on smart contracts and Ethereum platforms while noting that chain-of-custody semantics and evaluation methods remain inconsistently formalized [6].

Two problems remain open in this setting. First, authenticity is usually established at lot or batch granularity, whereas cloning and substitution happen per pack. Authenticating a single pack requires either one on-chain record per pack, whose cost grows with production volume, or a verification step tied to a state that records whether the pack was already checked. Second, a recall is not complete when holders are notified: the system must also establish whether each handler acted before a deadline and how many packs remain unaccounted for.

This paper presents *RecallChain*, a hybrid on-chain/off-chain framework that extends a role-based batch pipeline (supplier, manufacturer, distributor, seller) with three mechanisms. Its contributions are:

- 1) **Batch-level commitment with unit-level authentication.** Each pack carries a serial number and a random secret under a scratch layer. Only one Merkle root per batch is stored on-chain, and the contract itself verifies proofs, so a relay server can withhold service but cannot forge a genuine verdict.
- 2) **Wallet-free, anonymous unit claims.** The first valid check binds a pack to a key generated on the patient's device through a signed, relayed transaction. The resulting per-pack state makes a second scan of the same code a clone signal, and a claim made before the batch reaches a seller is flagged as possible diversion.
- 3) **Accountable recall.** Only the creating manufacturer can recall a batch. Every handler must quarantine its packs and confirm before an on-chain deadline; anyone

may escalate a handler who misses it, and a closure report accounts for every pack of the batch.

4) **A candid evaluation.** We report a functional test suite and a gas snapshot together with the governance parameters, scope, workload, measurement conditions and evidence artifacts that [6] recommend, a threat model, and the residual risks and missing experiments.

We make no claim of a new cryptographic primitive: Merkle commitments [7], signed messages and role-based smart-contract access control are established techniques. Their contribution is their composition for unit-level authentication and recall accountability, and an explicit account of what that composition does and does not guarantee. The prototype does not implement statistical or machine-learning anomaly detection; the rule-based integrity indicators it does implement are described as such in Section III-F.

II. LITERATURE SURVEY

A. Blockchain Traceability in the Pharmaceutical Supply Chain

Zakari et al. [5] reviewed 38 peer-reviewed studies from 2016 to 2021 and identified counterfeit-drug prevention, product distribution, tracking and tracing, and safety and security as the principal application domains. They list scalability limits, standardization gaps and the lack of universal implementation guidelines among the open challenges. Díaz et al. [6] mapped 103 records from 2019 to 2025 into implemented prototypes, architectural frameworks and adoption-barrier studies, and called for benchmark-oriented assessments and minimal reporting standards that specify governance parameters, logistics scope, workload, measurement conditions and evidence artifacts.

Musamih et al. [8] proposed an Ethereum framework for drug traceability that uses Solidity smart contracts and IPFS for off-chain documents and images. They evaluated four contract functions (lot creation, sale authorization, whole-lot purchase and individual box transactions) by gas consumption and performed a security analysis including vulnerability scanning with SmartCheck and Oyente. It is the closest design to ours in combining Ethereum smart contracts with off-chain storage, and we follow its evaluation style of gas measurement plus security analysis.

B. Product Authentication and Consumer-Facing Verification

Toyoda et al. [9] addressed counterfeits in the post-supply-chain stage with a blockchain-based product ownership management system. Rahman et al. [10] combined EdDSA signatures with QR-code verification for real-time consumer authentication of drugs and compared throughput, execution time and latency against RSA and ECDSA variants; they acknowledge that the framework assumes certified manufacturers supply genuine drugs and lacks live-product tracking. Finke et al. [11] describe SPOQchain, a platform for lifecycle traceability and originality verification that uses a batching mechanism to remove lifecycle redundancies and emphasizes user privacy.

C. Recall Management

Agrawal et al. [12] present a Hyperledger-based drug-recall system with forward and backward recall workflows intended to reduce the time and cost of retrieving defective drugs. Sitaula et al. [13] describe ACTMeds, an Ethereum and React platform with role-based access control that prototypes a zero-knowledge recall mechanism, in which regulators prove that recall conditions meet policy requirements without disclosing sensitive incident details. They evaluate it with STRIDE threat modeling, static and dynamic security testing and zero-knowledge performance metrics and describe it as a controlled pilot.

D. Cross-Cutting Concerns

Three concerns from outside the pharmaceutical literature shape our design and limitation analysis. The *oracle problem* [14] means that a blockchain guarantees the integrity of what it records, not the truth of off-chain facts; the quarantine counts reported by recall holders in our system are such facts. *Front-running* [15] arises because pending transactions are visible and can be copied or reordered; a claim transaction that carries a scratch secret is exposed to it. Finally, surveys of *anomaly detection* [16] distinguish point, contextual and collective anomalies and rest on explicit data models and detection metrics. The deterministic indicators in our prototype do not meet that bar, and we do not present them as anomaly detection.

E. Research Gap

The reviewed works establish that smart contracts can record custody events [5], [6], [8], that signatures and QR codes can support consumer-side authentication [10], [11], and that recall workflows [12] and privacy-preserving recall proofs [13] are feasible. Within the works we could verify, we did not find a design that combines, in one contract-enforced mechanism, (i) a constant-size on-chain commitment to per-pack secrets, (ii) a per-pack on-chain claim state that converts repeated verification into a clone and diversion signal without storing patient identity, and (iii) a deadline-enforced handler acknowledgement protocol with a per-batch accounting identity. In addition, [6] reports that evaluation reporting is inconsistent across studies.

The contribution of RecallChain is therefore not a new algorithm but the integration of these three mechanisms in one framework and a transparent account of its guarantees. Our survey is not systematic, and the absence of such a combination in it is not evidence of its absence in the wider literature.

III. PROPOSED SYSTEM

RecallChain tracks a medicine batch from creation to sale and lets a patient check an individual pack without a cryptocurrency wallet. Six parties interact: the *owner* (contract deployer), who registers actors; four registered roles, *supplier*, *manufacturer*, *distributor* and *seller*; and the *patient*. A batch moves through a strict pipeline, Created → Processing → In Transit → For Sale → Sold, and each transition is restricted to the corresponding role.

A. System Architecture

The system has three tiers (Fig. 1). A Solidity smart contract is the system of record. An Express relay server stores leaf

hashes, builds Merkle proofs, answers read-only verification requests and relays patient claims. A Next.js web client offers staff pages, which sign transactions through MetaMask, and a patient page that needs no wallet. Table I lists the modules.

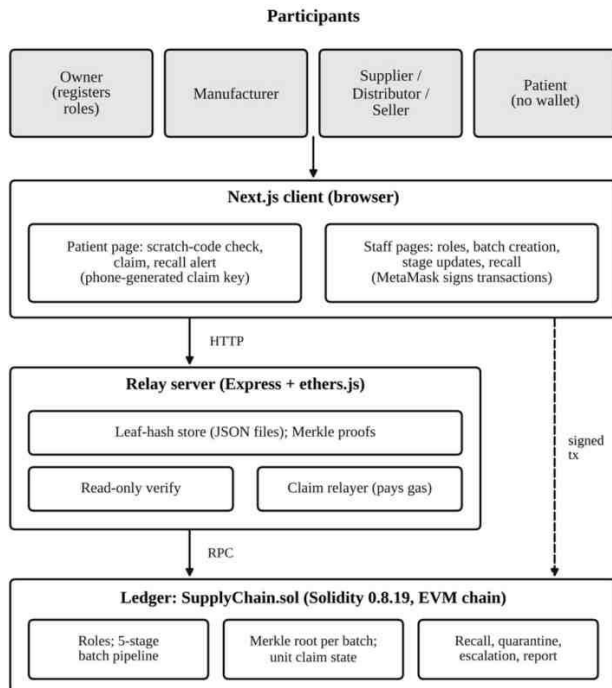


Fig. 1: RecallChain System Architecture

Module	Main function
Role registry	Owner-only registration of supplier, manufacturer, distributor and seller wallets
Batch lifecycle	Five-stage pipeline with role-gated transitions; batch quantity and Merkle root recorded at creation
Scratch-code authentication	Contract-side verification of a secret against the batch root
Unit claim	First-claim-wins state per pack; relayed claims bound to the patient's signature
Recall workflow	Manufacturer-only recall, quarantine, acknowledgement and escalation
Closure report	Per-batch accounting of quarantined, claimed and unaccounted packs
Relay server	Leaf-hash store, proofs, read-only verification, claim relay, provenance timeline

Table I — System Modules and Functions

We use “hybrid” in the data-architecture sense: integrity-critical commitments and state are on-chain, whereas bulk data and secret-adjacent data are kept off-chain. We do not use it in the public/private-chain sense, and the prototype has been run only on a local development chain. Table II shows where each datum lives. No single store holds every secret: the relay holds only leaf hashes, and the secrets exist only on the printed codes (and the manufacturer's CSV file) and, transiently, in the request that checks them.

Location	Data held
On-chain	Batch quantity and Merkle root; role registry; stage; per-pack claim or quarantine state; recall state, deadline and holder statuses; reason hash
Relay (JSON files)	Leaf hashes per batch; no scratch secrets at rest
Manufacturer browser / printed CSV	Scratch secrets (generated with <code>crypto.getRandomValues</code>)
Patient device	Claim key pair and list of claimed packs (browser storage)

Table II — Where Each Datum Lives

B. Threat Model

The assets are the authenticity verdict for each pack, the integrity of custody and recall records, and patient anonymity. We consider five adversaries: (A1) a counterfeiter with no access to printed codes; (A2) a counterfeiter who copies a code from a genuine pack or a leaked file; (A3) a curious or malicious relay operator; (A4) a registered handler that neglects or misreports a recall; and (A5) an observer of pending transactions. We assume that the owner and manufacturer keys are not compromised, that the printed code list is protected like a password list, and that the scratch layer hides the secret until consumer use. Actions in the physical world are not verified by the chain. Table III summarizes mitigations and residual risks; the residual-risk column is our analysis of the contract and relay, not an experimental result.

Threat	Mitigation implemented	Residual risk
Fake pack, guessed code (A1)	256-bit secret; contract checks Merkle proof against on-chain root	Negligible guessing probability
Copied genuine code (A2)	First check claims the pack; later checks show “already used”	Cannot tell which physical pack is genuine; first scanner wins
Lying relay (A3)	Verdict recomputed by the contract; relay cannot forge “Genuine”	Relay sees secrets at check and claim time; can withhold service or claim for itself
Diversion before sale (A2)	Claim before the batch reaches a seller is flagged	Flag only; no prevention
Ignored recall (A4)	Deadline, permissionless escalation, permanent miss counter	Quarantine actions are self-reported
Unauthorized stage change or recall	Role checks: manufacturer-only recall (tested)	Owner and manufacturer keys are trusted
Replayed relayed claim	Signed digest binds contract, chain ID, batch, serial and claimant	None identified
Front-running a claim (A5)	Not mitigated	Secret visible in calldata on a public chain

Table III — Threat Model Summary

C. Batch Commitment and Scratch-Code Authentication

When the manufacturer creates a batch of n packs, the browser draws a random 32-byte secret s_i for each serial $i = 1 \dots n$ and computes the leaf

$$\text{leaf}(i) = \text{keccak256}(\text{keccak256}(\text{abi.encode}(i, s_i)))$$

The double hash follows the usual leaf convention that prevents an internal tree node from being presented as a leaf. The leaves are folded into a Merkle tree [7] whose 32-byte root is passed to the contract with the batch. The leaf hashes, but never the secrets, are uploaded to the relay, which

accepts them only if they rebuild the on-chain root. The manufacturer downloads a CSV of serial and secret pairs for the printer.

To check a pack, the relay locates the serial whose leaf the submitted secret reconstructs, builds its Merkle proof and calls the contract's read-only function, which recomputes the leaf and verifies the proof against the stored root (Fig. 2). Four verdicts are possible: Invalid, Genuine, AlreadyClaimed and Recalled. The relay cannot make an invalid code verify, because the check is repeated inside the contract. A code verifies only if the attacker finds a second preimage of Keccak-256 or guesses a secret, which for a uniformly random 256-bit secret has probability of about $n/2Z^{56}$ per guess. Knowing a neighboring leaf, which appears in proof, does not allow claiming the neighbor (tested). The on-chain cost of the commitment is one 32-byte root regardless of n , and a proof has about $\log_2 n$ hashes.

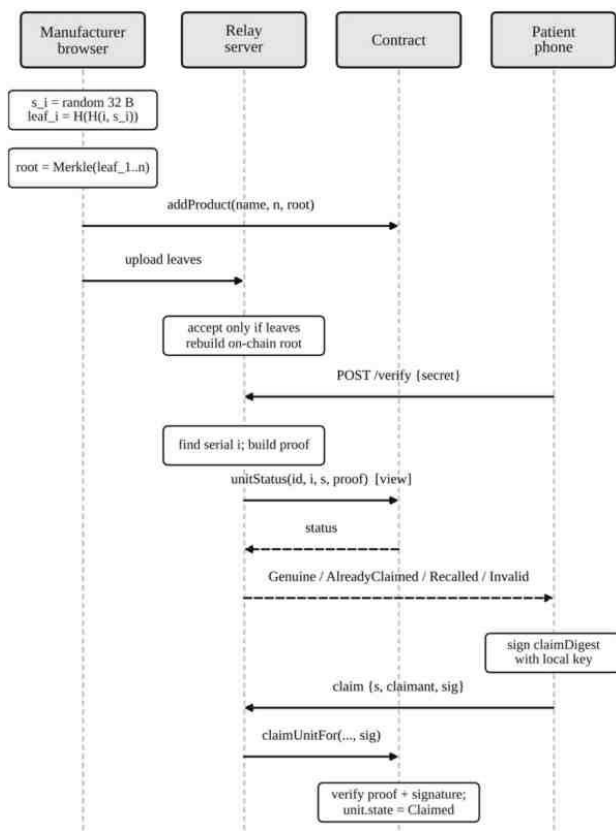


Fig. 2: Batch Registration, Scratch-Code Verification and Relayed Claim

D. Unit Claim and Clone Signal

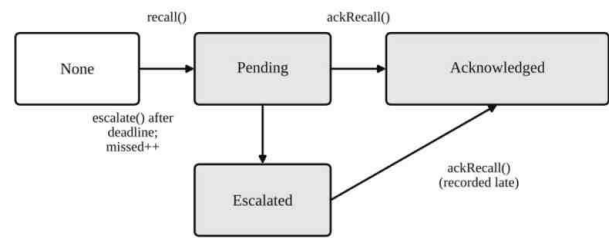
After a Genuine verdict the patient may claim the pack. The device generates its own key pair, signs the contract-provided digest, which commits to the contract address, chain ID, batch, serial and claimant, with an EIP-191 personal signature, and the relay submits the transaction and pays the gas. The contract verifies the proof and the signature and stores the claimant key; no name, phone number or e-mail address goes on-chain. A patient who prefers not to reveal the secret to the relay can instead call the claim function from their own wallet. A pack is in exactly one of three states: None, Claimed or Quarantined. A second check of a claimed code returns AlreadyClaimed, which is the clone signal, and a claim is flagged

claimedBeforeSale if the batch had not reached the For Sale stage or had no seller.

E. Recall with Confirmation and Closure Report

Only the manufacturer that created a batch can recall it, and a recall is permanent because the prototype has no regulator role that could safely lift one. A recall records a hash of the reason, sets the deadline to the recall time plus a configurable window (default 24 hours; the owner may shorten it to a minimum of 60 seconds for demonstrations) and registers every actor that handled the batch as a *holder* with status Pending; a wallet holding several roles counts once (Fig. 3). A recalled batch is frozen: stage changes and claims revert, and every valid scan reports Recalled.

(a) Holder status for a recalled batch



(b) Unit (pack) state

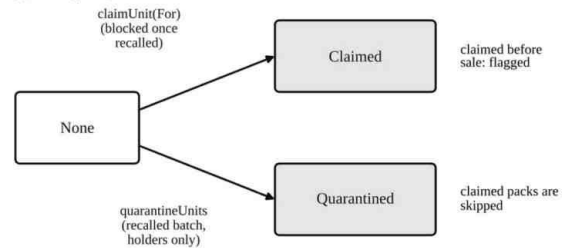


Fig. 3: Holder Status and Pack State Transitions

A holder lists the serials it removed from stock (at most 500 per transaction; packs already claimed are skipped) and then confirms. After the deadline anyone may escalate a holder that has not confirmed, which increments a permanent per-address miss counter; a late confirmation is still accepted and marked late. The closure report returns the quantity, the numbers of quarantined and claimed packs, the number claimed before sale, the number unaccounted for, and the counts of holders that acknowledged, were escalated or are pending. Because a pack is in at most one state, $quarantined + claimed + unaccounted = quantity$ always holds.

F. Rule-Based Integrity Indicators

The framework implements four deterministic indicators: a repeated scan of a claimed code (AlreadyClaimed), a claim before sale, a holder escalation with its permanent miss counter, and the skipping of already-claimed packs during quarantine. They are rules evaluated by the contract. They have no learned parameters, no tunable thresholds and no reported false-positive or false-negative rates, so we call them integrity indicators and not anomaly detection. Statistical anomaly detection over the event history is future work (Section VI).

IV. METHODOLOGY & IMPLEMENTATION

We followed a build-and-evaluate approach. Requirements were derived from the gaps identified in Section II; the contract, relay and client were implemented; the authorization rules, state transitions and accounting identities were asserted in an automated test suite on a local network; gas costs were measured with a gas reporter; and a threat review of the contract and relay was carried out by inspection (Table III).

A. Technology Stack

The contract is written in Solidity 0.8.19 (optimizer enabled, 200 runs) and uses OpenZeppelin Contracts 4.9.6 for Merkle-proof verification and ECDSA recovery. Hardhat 2.27.1 provides compilation, tests and local deployment. The relay is a Node.js service using Express 4 and ethers 6; the client uses Next.js 16, React 19, web3.js 4, an OpenZeppelin Merkle-tree library for proof construction and qrcode.react for QR generation. The prototype codebase is named MedChain; RecallChain denotes the framework described in this paper.

B. Contract Implementation

The pipeline retains the role-and-stage model and hardens it. Role lookup uses a nested mapping from address and role to actor identifier, replacing linear scans with constant-time reads; the creating manufacturer is recorded at batch creation; only that manufacturer may advance its batch from Processing; a second seller can no longer overwrite a listed batch; and a wallet may hold several roles but cannot register twice in one role. Batch creation requires all four roles to exist, a quantity from 1 to 1,000,000 and a non-zero root. These changes are engineering corrections of the base flow and are not claimed as research contributions.

C. Off-Chain Components

Scratch secrets are generated in the manufacturer's browser with the Web Crypto random generator, and the interface limits a batch to 10,000 packs. The relay validates an uploaded leaf list before storing it: every leaf must be a 32-byte hex string, leaves must be unique, their count must equal the batch quantity, and the rebuilt root must equal the on-chain root. Files are written atomically and stored only as leaf hashes. For a check, the relay either looks only in a given batch or, if no batch is given, searches the uploaded batches from the most recent; the serial search is linear in batch size. For a claim, the relay first performs a dry-run call so that the patient receives the contract's revert reason, and then submits the transaction through a serialized queue to keep nonces ordered.

D. QR Code and Patient Flow

The QR code printed or shown for a batch encodes only a link to the verification page with the batch identifier. It is for navigation: it carries no secret and does not by itself authenticate a pack, which is the role of the scratch code. The patient page accepts the code, shows the verdict and the batch provenance, and offers the claim. The device stores the claim key and the list of claimed packs in browser storage and re-checks them every 30 seconds while the page is open, raising a recall alert if a claimed pack's batch has been recalled.

E. Provenance Timeline

The relay reconstructs a public timeline from the contract's product-creation and stage-update events: for each event it reports the stage, the block timestamp, the transaction hash and the role of the sender, determined by matching the sender address against the registered actors of the batch.

F. Implemented Versus Planned

Table IV separates what is implemented from what is not. "Tested" means covered by the automated contract suite; relay and client behaviour has no automated tests in the code base examined.

Capability	Status
Role registry; five-stage pipeline	Implemented; tested
Merkle-root commitment; scratch-code verification	Implemented; tested
Relayed anonymous claim (EIP-191)	Implemented; contract tested
Recall, quarantine, escalation, closure report	Implemented; tested
Rule-based integrity indicators	Implemented; tested; no detection-rate evaluation
Provenance timeline from events	Implemented; not automatically tested
QR link to verification page	Implemented; navigation only
Patient recall alert (30 s re-check)	Implemented; not automatically tested
Statistical or ML anomaly detection	Not implemented
Commit-reveal claims (front-running defence)	Not implemented
Multi-seller distribution; unit-level custody	Not implemented
Regulator role; recall reversal	Not implemented
Public testnet deployment; scale benchmarks	Not done

Table IV — Implementation Status

Code availability: [repository URL and commit identifier to be added by the authors].

V. RESULTS AND LIMITATIONS

The prototype was evaluated to verify the correctness of the authorization rules, state transitions and recall accounting, and to measure the gas cost of its operations. All results come from a local Hardhat network. They are functional and cost results and not performance results, and they should not be read as evidence of detection capability or production scalability.

A. Experimental Setup and Reporting Card

Following [6], Table V states the governance parameters, scope, workload, measurement conditions and evidence artifacts of the evaluation.

Parameter	Value
Governance	Single owner registers all actors; manufacturer-only, permanent recall; recall window 24 h by default (minimum 60 s)
Scope	Four roles, five stages; one supplier, manufacturer, distributor and seller per batch
Workload	Test fixtures of 1 to 1,000 packs; a 200-serial quarantine
Conditions	Hardhat network, chain ID 1337, 30,000,000 block gas limit; Node.js 22.22.2; Hardhat 2.27.1; Solidity 0.8.19; OpenZeppelin 4.9.6; run of 2026-10-01
Evidence	Contract source; test file with 33 tests; captured test and gas record; gas-reporter averages over successful transactions; interface screenshots (Figs. 4–7)

Table V — Evaluation Reporting Card

B. Functional Evaluation

The suite of 33 tests passed with no failures (Table VI). The tests cover the base flow, the foundation fixes, scratch-code verification, unit claims, recall and the closure report. In the base flow, owner-only registration, required roles, manufacturer-only creation, the complete stage sequence, and rejection of wrong-role and out-of-order calls were asserted.

Group	Tests	Behaviour asserted
Base flow	8	Ownership; registration; roles; stage order; rejection of wrong-role and out-of-order calls
Foundation fixes	6	Manufacturer, quantity and root recorded; invalid inputs rejected; no duplicate role; single seller
Scratch-code verification	5	Contract and off-chain leaf formulas agree; wrong secret, serial or batch is Invalid; neighbour leaf cannot be claimed
Unit claim	4	First claim wins; relayed claim with signature; bad signature rejected; claim before sale flagged
Recall	7	Manufacturer-only; batch frozen; holders; quarantine rules; deadline, escalation and late confirmation; window control
Closure report	3	Accounting identity; non-recalled batch; 1,000-pack batch with proof length ≤ 10
Total	33	33 passed, 0 failed

Table VI — Functional Evaluation of the Smart Contract

C. Authentication and Recall Evaluation

For authentication, a genuine code returned Genuine, whereas a wrong secret, wrong serial, out-of-range serial, unknown batch or a code taken from another batch returned Invalid. A relayed claim succeeded only when the signature matched both the claimant and the serial. A repeated claim reverted and the code then reported AlreadyClaimed. A claim made before the batch reached a seller was counted in the claimed-before-sale total, and a later claim was not.

For recall, owner, other manufacturers and sellers could not recall the batch; after a recall, stage changes and claims reverted and every valid scan reported Recalled; a code from outside the batch still reported Invalid. Escalation reverted before the deadline and succeeded after it, the miss counter increased, and a late confirmation was recorded as late. In a 10-pack scenario the closure report returned 4 quarantined,

3 claimed (1 before sale) and 3 unaccounted packs, and the identity held. These are assertions about contract behaviour under scripted inputs; they say nothing about handlers' real-world conduct.

D. Gas Cost

Table VII gives the averages reported by the gas reporter over successful transactions. Call counts are small, so the figures are a snapshot, not a benchmark. Batch creation costs 204,006 gas and is independent of the number of packs because only the root is stored. Relative to the original contract's 139,493 gas, this is an increase of about 46%, the price of storing the quantity and root; the original-contract deployment cost was 1,701,274 gas against 3,476,363 now (11.6% of the block limit). The claim costs 89,220 gas on average (range 68,841–111,829) and the relayed claim 92,463, so the signature check adds little. Quarantine costs about 23,000 gas per serial: 200 serials used 4,633,914 gas, 15.4% of the block limit. By linear extrapolation, a 500-serial transaction would need about 11.6 million gas, which fits the test network's limit but could be costly on a public chain.

Operation	Avg. gas	Note
Contract deployment	3,476,363	
Role registration (each)	$\approx 185,000$ – 187,000	
addProduct (root)	204,006	Constant in n
supplyProduct	74,336	
processProduct	37,318	
distributeProduct	57,207	
listForSale	54,274	
markProductSold	37,261	
claimUnit (own wallet)	89,220	68,841–111,829
claimUnitFor (relayed)	92,463	
recall	235,463	173,628– 315,726
quarantineUnits	$\approx 23,000$ per serial	200 serials: 4,633,914
ackRecall	48,570	
escalate	63,228	
setRecallWindow	29,898	

Table VII — Gas Snapshot on the Local Hardhat Network

E. Interface Evidence

Figs. 4–7 are screenshots of the running prototype from a single demonstration session on the local Hardhat network (port 8545) with MetaMask. They show that the contract rules are enforced and surfaced through the interface; they are qualitative illustrations, not test results, and the batch names are arbitrary placeholders.

Fig. 4 shows the access check on the role-registration page: a connected wallet that is not the contract owner is told that only the owner can register participants, and the interface displays the contract owner's address next to the connected account. Fig. 5 shows the on-chain batch list with the pipeline stages; batch 1 carries the "Recalled — frozen" marker while still in the Created stage. Fig. 6 shows the closure report for that recalled batch: of 10 packs, 1 was quarantined by holders, 0 were with customers and 9 were unaccounted for, which satisfies the accounting identity, and

the page displays the remaining confirmation time (22 h 44 min). The recall form states a confirmation window of 1,440 minutes, the 24-hour default, and that only a hash of the reason is stored on-chain. Fig. 7 shows the patient page, which needs no wallet: a single scratch-code field, a list of claimed packs that is re-checked for recalls every 30 seconds while the page is open, and a recall lookup by batch and serial that does not need the code.



Fig. 4: Owner-Only Role Registration Enforced in the Interface

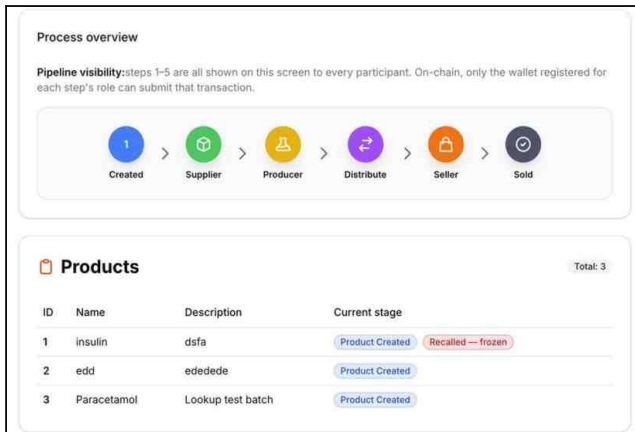


Fig. 5: Pipeline Stages and On-Chain Batch List, With a Recalled Batch

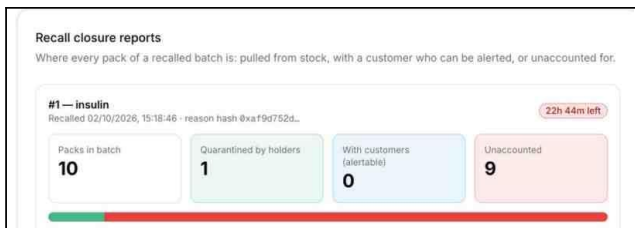


Fig. 6: Recall Closure Report for Batch 1 (10 Packs)

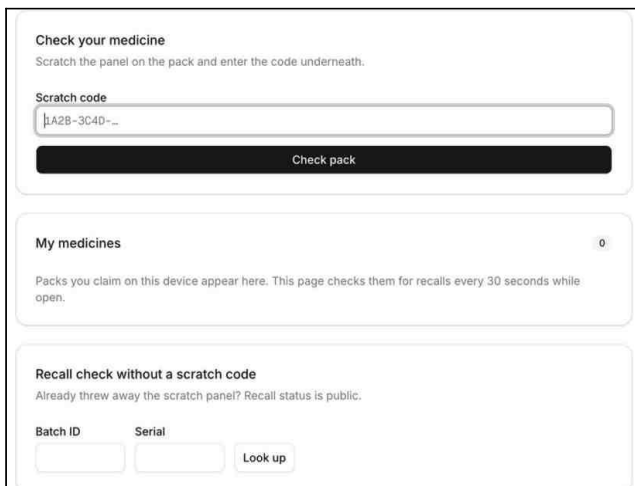


Fig. 7: Wallet-Free Patient Verification Page

F. Discussion

The results show that the specified authorization rules, state transitions and accounting identities behave as asserted on a local network, and that commitment cost is constant per batch while per-pack operations cost tens of thousands of gas. They support the architectural claim that a single root

can authenticate individual packs without storing one record per pack. They do not show lower cost than a per-pack design, because no such baseline was measured, nor that the clone and diversion indicators detect real counterfeiting, nor that recall coverage in the physical world matches what handlers report.

G. Limitations

Evaluation scope. All evidence comes from a local in-process network with no network latency, mempool contention or finality delay. Latency, throughput and relay scaling were not measured, and the fixtures contain at most 1,000 packs, whereas the contract permits 1,000,000. The relay's demonstration store holds four batches of at most ten packs. There is no baseline comparison and no code-coverage or static-analysis report. Beyond the suite, the project record mentions 20 API and 31 browser end-to-end checks; their scripts are not part of the code base we examined, so we treat them as supplementary and do not tabulate them.

Secrets and the relay. By inspection, the scratch secret is sent to the relay on every verification request as well as on a claim, so a malicious relay could claim a pack for itself. A direct claim from the patient's wallet avoids the relay but places the secret in transaction calldata, where an observer of pending transactions on a public chain could copy it and claim first [15]. We have not demonstrated this attack; a commit–reveal claim is the natural remedy and is not implemented. The serial search in the relay is linear in batch size and has not been tested at scale.

Recall accounting. Quarantine is a self-report [14]. By inspection, the contract checks only that the caller is a holder and that the serial is in range, so any holder can mark any unclaimed serial of a recalled batch as quarantined, and the first report wins. The closure report therefore measures reported coverage and not verified coverage, and “unaccounted” can be reduced by false reports. Patients who never claim cannot be alerted.

Custody and governance. Each batch has one supplier, manufacturer, distributor and seller, so splitting a batch across many pharmacies is not supported. Stage functions check the caller's role but not that the batch was assigned to that actor, and no custody is recorded per pack. The owner registers all actors, a single manufacturer key can recall its batches permanently, and there is no regulator role or recall reversal. Product names, descriptions and actor details are public on-chain.

Authentication semantics. The scheme cannot tell which of two physical packs carrying the same code is genuine, and a thief who scratches a pack before sale holds the claim; both are flagged by later scans but not prevented. A leaked code list defeats authentication for its batch. The QR code does not authenticate. No statistical anomaly detection is implemented, and the indicators of Section III-F have no measured detection rates.

VI. FUTURE WORK AND CONCLUSION

A. Future Work

The following are planned and not implemented. (1) *Evaluation:* measure gas against batch size and proof length;

compare against a per-pack on-chain registration baseline; measure relay latency and memory against batch size; run an attack matrix covering cloned codes, wrong-batch codes, replayed claims and claim front-running; run static analysis and report coverage; and deploy to a public test network for realistic cost and finality. (2) *Anomaly detection*: define an anomaly model over the event history (for example scan timing, claim-before-sale rates and handler response latency), generate labelled event streams from a documented simulation, and compare the rule-based indicators with statistical detectors using precision, recall and F1 [16]. (3) *Protocol*: commit–reveal claims, multi-seller distribution with unit-level custody transfer, a regulator role with governed recall reversal, and physical spot-check evidence for quarantine reports. (4) *Alignment*: encode the data elements required by the Indian rule [4] and by serialization standards, and study the privacy benefit of zero-knowledge techniques [13]. (5) *Usability*: a user study of the patient flow.

B. Conclusion

This paper presented RecallChain, a hybrid on-chain/off-chain framework that combines a constant-size Merkle commitment for unit-level scratch-code authentication, wallet-free anonymous claims that expose cloned codes and pre-sale diversion, and a deadline-enforced recall protocol with an accounting identity. The contract and its 33 tests passed on a local network, and the gas snapshot shows constant-cost batch registration and per-claim costs below 100,000 gas.

The evidence is limited to functional behaviour and cost on a local chain. The system does not yet include anomaly detection, protection against claim front-running, verification of recall actions in the physical world or multi-seller distribution, and its field performance is unknown. The planned evaluation and protocol extensions above are required before the framework can be considered for deployment.

References

- [1] World Health Organization, “Substandard and falsified medical products,” Fact sheet, 3 Dec. 2024. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/substandard-and-falsified-medical-products>
- [2] U.S. Congress, “Drug Quality and Security Act, Title II: Drug Supply Chain Security Act,” Pub. L. No. 113-54, 2013. See also U.S. Food and Drug Administration, “Drug Supply Chain Security Act (DSCSA).” [Online]. Available: <https://www.fda.gov/drugs/drug-supply-chain-integrity/drug-supply-chain-security-act-dscsa>
- [3] European Commission, “Commission Delegated Regulation (EU) 2016/161 of 2 October 2015 supplementing Directive 2001/83/EC of the European Parliament and of the Council by laying down detailed rules for the safety features appearing on the packaging of medicinal products for human use,” Official Journal of the European Union, 9 Feb. 2016.
- [4] Ministry of Health and Family Welfare, Government of India, “Drugs (Eighth Amendment) Rules, 2022,” G.S.R. 823(E), The Gazette of India, Extraordinary, Part II, Sec. 3(i), 17 Nov. 2022.
- [5] N. Zakari, M. Al-Razgan, A. Alsaadi, H. Alshareef, H. Al saigh, L. Alashaikh, M. Alharbi, R. Alomar, and S. Alotaibi, “Blockchain technology in the pharmaceutical industry: a systematic review,” *PeerJ Comput. Sci.*, vol. 8, Art. no. e840, 2022, doi: 10.7717/peerj-cs.840.
- [6] F. Díaz, N. Cerna, R. Liza, B. Motta, and S. Rojas-Flores, “Blockchain-enabled traceability in pharmaceutical supply chains: A mapping review of evidence for visibility, anti-counterfeiting, and chain-of-custody control,” *Logistics*, vol. 10, no. 4, Art. no. 85, 2026, doi: 10.3390/logistics10040085.
- [7] R. C. Merkle, “A digital signature based on a conventional encryption function,” in *Advances in Cryptology—CRYPTO ’87*, Lecture Notes in Computer Science, vol. 293. Berlin, Germany: Springer, 1988, pp. 369–378, doi: 10.1007/3-540-48184-2_32.
- [8] A. Musamih, K. Salah, R. Jayaraman, J. Arshad, M. Debe, Y. Al-Hammadi, and S. Ellahham, “A blockchain-based approach for drug traceability in healthcare supply chain,” *IEEE Access*, vol. 9, pp. 9728–9743, 2021, doi: 10.1109/ACCESS.2021.3049920.
- [9] K. Toyoda, P. T. Mathiopoulou, I. Sasase, and T. Ohtsuki, “A novel blockchain-based product ownership management system (POMS) for anti-counterfeits in the post supply chain,” *IEEE Access*, vol. 5, pp. 17465–17477, 2017, doi: 10.1109/ACCESS.2017.2720760.
- [10] M. S. Rahman, N. Nahar, M. H. Imam, M. N. Bhuyian, M. A. Rahman, M. U. Khandaker, and S. F. Ahmed, “A blockchain-based framework for drug security: Leveraging EdDSA to prevent counterfeiting,” *Array*, vol. 28, Art. no. 100604, 2025, doi: 10.1016/j.array.2025.100604.
- [11] M. Finke, A. Dmitrienko, and J. Stang, “SPOQchain: Platform for secure, scalable, and privacy-preserving supply chain tracing and counterfeit protection,” *arXiv:2408.17049*, 2024.
- [12] D. Agrawal, S. Minocha, S. Namasudra, and A. H. Gandomi, “A robust drug recall supply chain management system using hyperledger blockchain ecosystem,” *Comput. Biol. Med.*, vol. 140, Art. no. 105100, 2022, doi: 10.1016/j.compbiomed.2021.105100.
- [13] A. Sitaula, M. A. Uddin, J. Ayoade, N. H. Chu, and R. Rafeh, “Zero-knowledge proof-based privacy-preserving pharmaceutical traceability and recall using blockchain,” *Blockchains*, vol. 4, no. 2, Art. no. 5, 2026, doi: 10.3390/blockchains4020005.
- [14] G. Caldarelli, “Understanding the blockchain oracle problem: A call for action,” *Information*, vol. 11, no. 11, Art. no. 509, 2020, doi: 10.3390/info11110509.
- [15] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels, “Flash Boys 2.0: Frontrunning, transaction reordering, and consensus instability in decentralized exchanges,” *arXiv:1904.05234*, 2019.
- [16] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Comput. Surv.*, vol. 41, no. 3, 2009, doi: 10.1145/1541880.1541882.